บทที่ 6 พีชคณิตเชิงสัมพันธ์และแคลคูลัสเชิงสัมพันธ์

Relational Algebra & Relational Calculus

วัตถุประสงค์

- สามารถอธิบายถึงความเกี่ยวข้องของพีชคณิตเชิงสัมพันธ์และแคลคูลัสเชิงสัมพันธ์ กับการจัดการฐานข้อมูลเชิงสัมพันธ์ได้
- สามารถอธิบายการดำเนินการกับรีเลชันด้วยพีชคณิตเชิงสัมพันธ์ได้
- สามารถสร้างรีเลชันที่มีข้อมูลตามที่ต้องการโดยใช้ตัวดำเนินการพีชคณิตเชิงสัมพันธ์ได้
- สามารถอธิบายการดำเนินการกับรีเลชันด้วยแคลคูลัสเชิงสัมพันธ์แบบทูเพิลและสร้างรีเลชันจากแคลคูลัสเชิงสัมพันธ์แบบทูเพิลได้
- สามารถอธิบายการดำเนินการกับรีเลชันด้วยแคลคูลัสเชิงสัมพันธ์แบบโดเมนและสร้างรีเลชันจากแคลคูลัสเชิงสัมพันธ์แบบโดเมนได้

คำสำคัญ: พีชคณิตเชิงสัมพันธ์ (relational algebra); แคลคูลัสเชิงสัมพันธ์ (relational calculus); ภาษาสอบถาม (query language); ภาษาเชิงกระบวนคำสั่ง (procedural language); ภาษาเชิงไร้กระบวนคำสั่ง (non-procedural language); บริบูรณ์เชิงสัมพันธ์ (relationally complete); ซีเลคชัน (selection); โปรเจคชัน (projection); ผลคูณคาร์ทีเซียน (Cartesian product); ยูเนียน (union); ผลต่างของเซต (set difference); การเชื่อม (join); อินเตอร์เซกชัน (intersection); การหาร (division); แคลคูลัสเชิงสัมพันธ์แบบทูเพิล (tuple relational calculus); แคลคูลัสเชิงสัมพันธ์แบบโดเมน (domain relational calculus)

บทที่ 6 พีชคณิตเชิงสัมพันธ์และแคลคูลัสเชิงสัมพันธ์

Relational Algebra & Relational Calculus

6.1 บทนำ

การแสดงและแก้ไขข้อมูลในฐานข้อมูลเชิงสัมพันธ์จำเป็นต้องอาศัยภาษาที่มีโครงสร้างรองรับแบบจำลองข้อมูลเชิงสัมพันธ์ เราเรียกภาษานี้ว่าภาษาสอบถาม (query language) ซึ่งใช้สำหรับแสดงและแก้ไขข้อมูลในระบบจัดการฐานข้อมูลเชิงสัมพันธ์ ในบทนี้อธิบายถึงพื้นฐานของภาษาสอบถามอันได้แก่ พีชคณิตเชิงสัมพันธ์ (relational algebra) และแคลคูลัสเชิงสัมพันธ์ (relational calculus) ที่ Codd ใช้อ้างอิงในการจัดการกับข้อมูลในฐานข้อมูลเชิงสัมพันธ์ พีชคณิตเชิงสัมพันธ์จัดเป็นพื้นฐานของภาษาสอบถามแบบ สำหรับภาษาสอบถามที่อยู่บนพื้นฐานของพีชคณิตเชิงสัมพันธ์และแคลคูลัสเชิงสัมพันธ์ที่นิยมมากที่สุดได้แก่ SQL ซึ่งอธิบายในบทถัดไป ภาษาเชิงกระบวนคำสั่ง (procedural language) หมายถึงภาษาที่สามารถสั่งเป็นลำดับขั้นตอน ในขณะที่แคลคูลัสเชิงสัมพันธ์เป็นพื้นฐานของภาษาสอบถามแบบ ภาษาไร้กระบวนคำสั่ง (non-procedural language) หมายถึงภาษาที่ไม่จำเป็นต้องสั่งเป็นลำดับขั้นตอนแต่สามารถได้ผลลัพธ์เช่นเดียวกัน ภาษาสอบถามใดๆ จะต้องมีความบริบูรณ์เชิงสัมพันธ์ (relationally complete) กล่าวคือหากแคลคูลัสเชิงสัมพันธ์สามารถสร้างรีเลชันใดๆ ได้ ภาษาสอบถามที่สร้างขึ้นต้องสามารถสร้างรีเลชันนั้นๆ ได้เช่นเดียวกัน

หัวข้อที่ 6.2 อธิบายถึงพีชคณิตเชิงสัมพันธ์ หัวข้อที่ 6.3 อธิบายถึงแคลคูลัสเชิงสัมพันธ์ซึ่งมีสองรูปแบบ ได้แก่แคลคูลัสเชิงสัมพันธ์แบบทูเพิล และแคลคูลัสเชิงสัมพันธ์แบบโดเมน ซึ่งได้อธิบายไว้ในหัวข้อที่ 6.3.1 และ 6.3.2 ตามลำดับ

6.2 พีชคณิตเชิงสัมพันธ์ (Relational Algebra)

พีชคณิตเชิงสัมพันธ์เป็นภาษาเชิงทฤษฎีที่ใช้ตัวดำเนินการกับรีเลชันหนึ่งรีเลชันหรือหลายรีเลชัน ผลลัพธ์ที่ได้คือรีเลชันเช่นเดียวกัน ซึ่งผลลัพธ์นี้สามารถนำมากระทำด้วยตัวดำเนินการได้อีกในลักษณะของการซ้อนกันของตัวดำเนินการเช่นเดียวกับการใช้ตัวดำเนินการทางคณิตศาสตร์ทั่วๆ ไป เช่น A + B - C เป็นต้น การที่ทั้งรีเลชันที่ถูกดำเนินการและผลลัพธ์ต่างก็เป็นรีเลชันนี้เป็นสมบัติของพีชคณิตเชิงสัมพันธ์ที่เรียกว่าสมบัติการปิด (closure property)

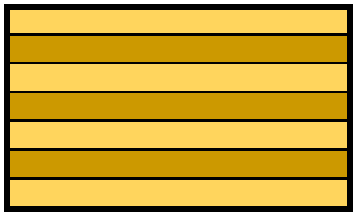
ตัวดำเนินการพีชคณิตเชิงสัมพันธ์ที่จะกล่าวถึงได้แก่ selection, projection, Cartesian product, union, set difference, join, intersection และ division ซึ่ง selection และ projection นั้นเป็นตัวดำเนินการแบบเอกภาค (unary operator) เนื่องจากกระทำบนรีเลชันเดียว ในขณะที่ตัวดำเนินการที่กระทำกับคู่ของรีเลชันจัดเป็นตัวดำเนินการแบบทวิภาค (binary operator) สำหรับ union, intersection, set difference และ Cartesian product เป็นตัวดำเนินการเซต

6.2.1 การดำเนินการเอกภาค (Unary Operations)

ตัวดำเนินการที่กระทำกับรีเลชันเพียงหนึ่งรีเลชัน ได้แก่ Selection และ Projection

6.2.1.1 ซีเลคชัน (Selection)

ซีเลคชัน (Selection) หรือ **เรสตริคชัน (Restriction)** คือ การกระทำบนรีเลชันหนึ่งรีเลชันในการสร้างรีเลชันใหม่ที่มีทูเพิลตรงตามเงื่อนไขที่กำหนด



รูปแบบกระทำกับรีเลชัน

$\sigma_{predicate}$(Relation)

ชื่อตาราง WHERE <CONDITION>

สัญลักษณ์การดำเนินการซีเลคชัน (บน) และ
รูปแบบ SQL (ล่าง) ที่สัมพันธ์กัน

σ คือตัวดำเนินการ selection และ predicate คือเงื่อนไขของการแสดงโดยกำหนดเงื่อนไขของค่าในคอลัมน์ใดๆ สัมพันธ์กับ SQL “WHERE” ที่จะกล่าวถึงในบทที่ 8

ตัวอย่าง Selection

ให้แสดงข้อมูลของนักศึกษาทั้งหมดที่มีเกรดตั้งแต่ 2.00 ขึ้นไป

$\sigma_{gpa >= 2.00}$**(Student)**

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	**3.44**
3100904032132	B5075688	สมศรี	somsri@com	18	**3.21**
3100905622132	B5075650	สมศรี	somsri@it	19	**3.82**
3100904055132	B5075831	สมศักดิ์	somsak@med	21	**1.80**
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	**2.00**

รีเลชัน ที่ถูกดำเนินการได้แก่รีเลชัน Student และ predicate คือ gpa >= 2.00 ตัวดำเนินการ selection สร้างรีเลชันที่บรรจุทูเพิลของนักศึกษาที่มีเกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป ซึ่งเป็นนักศึกษาทั้งหมดยกเว้นสมศักดิ์ ผลลัพธ์แสดงได้ดังนี้

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

นอกจากนี้เงื่อนไขยังสามารถมีความซับซ้อนหรือกำหนดเงื่อนไขหลายๆ เงื่อนไขได้ด้วยตัวดำเนินการทางตรรกะ $\wedge$ (และ) $\vee$ (หรือ) $\sim$ (นิเสธ)

ตัวอย่างที่ 6.1 Selection

ให้แสดงข้อมูลนักศึกษาทั้งหมดที่เกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป**และ**มีอายุมากกว่า 20 ปี

$\sigma_{gpa >= 2.00 \wedge age > 20}$ **(Student)**

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	**18**	**3.44**
3100904032132	B5075688	สมศรี	somsri@com	**18**	**3.21**
3100905622132	B5075650	สมศรี	somsri@it	**19**	**3.82**
3100904055132	B5075831	สมศักดิ์	somsak@med	**21**	**1.80**
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	**22**	**2.00**

พบว่านักศึกษาที่มีเกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไปคือนักศึกษาที่ชี้ด้วยเครื่องหมาย ⇨ ซึ่งมีจำนวน 4 คน ในขณะที่นักศึกษาที่มีอายุมากกว่า 20 ปี ชี้ด้วยเครื่องหมาย ➡ มีจำนวน 2 คน แต่นักศึกษาที่ทั้งมีเกรดเฉลี่ยตั้งแต่ 2.00 และมีอายุมากกว่า 20 ปีขึ้นไปมีเพียงคนเดียวได้แก่ สมน้ำหน้า จึงได้ผลลัพธ์เป็นรีเลชันดังนี้

cid	sid	name	login	age	gpa
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

ตัวอย่างที่ 6.2 Selection

ให้แสดงข้อมูลนักศึกษาทั้งหมดที่เกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป**หรือ**นักศึกษาที่มีอายุมากกว่า 20 ปี

$\sigma_{gpa >= 2.00 \vee age > 20}$ **(Student)**

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

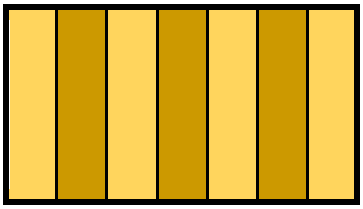
พบว่านักศึกษาที่มีเกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไปคือนักศึกษาที่ชี้ด้วยเครื่องหมาย ⇨ ซึ่งมีจำนวน 4 คน ในขณะที่นักศึกษาที่มีอายุมากกว่า 20 ปี ชี้ด้วยเครื่องหมาย ➡ มีจำนวน 2 คน ปรากฏว่านักศึกษาทั้งหมดที่มีคุณสมบัติเป็นไปตามเงื่อนไขข้อหนึ่งข้อใดหรือทั้งสองข้อ จึงได้ผลลัพธ์เป็นรีเลชันดังนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	**18**	**3.44**
3100904032132	B5075688	สมศรี	somsri@com	**18**	**3.21**
3100905622132	B5075650	สมศรี	somsri@it	**19**	**3.82**
3100904055132	B5075831	สมศักดิ์	somsak@med	**21**	**1.80**
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	**22**	**2.00**

6.2.1.2 โปรเจคชัน (Projection)

โปรเจคชัน คือ การกระทำบนรีเลชันหนึ่งรีเลชันในการสร้างรีเลชันใหม่ที่มีเฉพาะซับเซตในแนวตั้งหรือคอลัมน์ตามรายการแอททริบิวต์ที่ระบุ



รูปแบบกระทำกับรีเลชัน

$\Pi_{a1, \ldots, an}$(Relation)

ชื่อตาราง [ชื่อแอททริบิวต์ 1, … , ชื่อแอททริ

สัญลักษณ์การดำเนินการโปรเจคชัน (บน) และ
รูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

Π คือตัวดำเนินการ projection และ $a_1, \ldots, a_n$ รายการแอททริบิวต์ที่ต้องการแสดงจากรีเลชัน หรืออาจเขียนในรูปแบบชื่อตารางตามด้วยรายการแอททริบิวต์ที่ต้องการแสดง

ตัวอย่างที่ 6.3 Projection

ให้แสดงรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยของนักศึกษาทั้งหมด

$\Pi_{sid, name, gpa}$**(Student)**

Student ⇩ ⇩ ⇩

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

Projection สร้างรีเลชันใหม่จากรีเลชัน Student ที่ประกอบด้วยคอลัมน์ที่ระบุไว้คือ sid, name และ gpa ได้ผลลัพธ์ดังนี้

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

ผลลัพธ์ที่ได้จากตัวดำเนินการเชิงสัมพันธ์ยังคงเป็นรีเลชันซึ่งสามารถนำไปกระทำโดยตัวดำเนินการอื่นๆ ได้ ในกรณีนี้ผลลัพธ์ที่ได้จากตัวดำเนินการ projection สามารถนำไปดำเนินการต่อด้วยตัวดำเนินการ selection ได้ หรือสามารถสลับลำดับก่อนหลังได้ ทั้งนี้ระบบจัดการฐานข้อมูลจะเป็นผู้ทำการเลือกลำดับของตัวดำเนินการเพื่อให้การแสดงผลข้อมูลเกิดความรวดเร็วที่สุด ซึ่งเป็นข้อดีอันหนึ่งของระบบจัดการฐานข้อมูล

$\sigma_{condition}(\Pi_{a1, \ldots, an}(Relation))$

ชื่อรีเลชัน [ชื่อแอททริบิวต์ 1, …] WHERE <CONDITION>

หรือ

$\Pi_{a1, \ldots, an}(\sigma_{condition}(Relation))$

ชื่อรีเลชัน WHERE <CONDITION> [ชื่อแอททริบิวต์ 1, …]

ตัวอย่างที่ 6.4 Projection & Selection

ให้แสดงรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยของนักศึกษาที่มีเกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป

$\sigma_{gpa >= 2.00}(\Pi_{sid, name, gpa}(Student))$

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	**3.44**
3100904032132	B5075688	สมศรี	somsri@com	18	**3.21**
3100905622132	B5075650	สมศรี	somsri@it	19	**3.82**
3100904055132	B5075831	สมศักดิ์	somsak@med	21	**1.80**
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	**2.00**

Projection สร้างรีเลชันใหม่จากรีเลชัน Student ที่ประกอบด้วยคอลัมน์ที่ระบุไว้คือ sid, name และ gpa ได้ผลลัพธ์ดังนี้

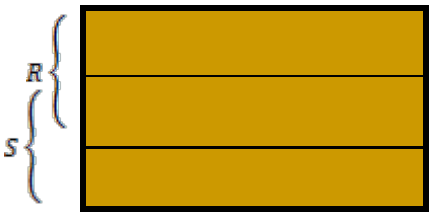
sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075832	สมน้ำหน้า	2.00

6.2.2 ตัวดำเนินการเซต (Set Operations)

การดำเนินการ selection และ projection กระทำกับรีเลชันเพียง 1 รีเลชันเท่านั้น แต่การใช้งานฐานข้อมูลโดยทั่วไปเรามีความจำเป็นต้องสร้างข้อมูลจากรีเลชันมากกว่า 1 รีเลชัน ในหัวข้อนี้จะได้อธิบายถึงการดำเนินการของเซตเริ่มด้วย union, intersection, set difference, Cartesian product, join และ division

6.2.2.1 ยูเนียน (Union)

ยูเนียน คือ การรวมรีเลชันเข้าด้วยกันโดยทูเพิลทั้งหมดในรีเลชันที่ถูกดำเนินการจะอยู่ในรีเลชันผลลัพธ์ ทูเพิลที่ซ้ำซ้อนกันจะถูกคงไว้เพียง 1 ทูเพิล และรีเลชันที่ยูเนียนกันต้องเข้ากันได้ (union-compatible)



รูปแบบกระทำกับรีเลชัน

Relation 1 ∪ Relation 2

ตารางที่ 1 UNION ตารางที่ 2

สัญลักษณ์การดำเนินการยูเนียน (บน) และ
รูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

∪ คือตัวดำเนินการ union สำหรับ R และ S หรือ Relation 1 และ Relation 2 เป็นรีเลชันที่จะรวมเข้าด้วยกัน ทั้งสองรีเลชันนั้นต้องเข้ากันได้โดยมีจำนวนแอททริบิวต์เท่ากัน ลำดับของแอททริบิวต์ตรงกัน และโดเมนของแต่ละแอททริบิวต์สัมพันธ์กัน

ตัวอย่างที่ 6.5 Union

R และ S เป็นรีเลชันที่มีข้อมูลของนักศึกษา **R ∪ S** แสดงได้ดังนี้

R

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

∪

S

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21

R ⇨

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

S ⇨

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21

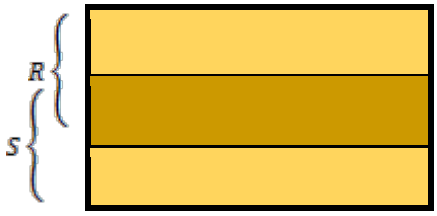
R ∪ S

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00
B5075688	สมศรี	3.21

R มีนักศึกษา 3 คน และ S มีนักศึกษา 2 คน การ union กันนั้นนำทูเพิลทั้งหมดมารวมกัน แต่ทูเพิลที่ซ้ำซ้อนกันที่ได้ทำเครื่องหมาย ⇨ ไว้นั้นได้แก่ทูเพิลของสมชาย ซึ่งเป็นนักศึกษาคนเดียวกัน สังเกตได้จากข้อมูลที่เหมือนกันโดยเฉพาะรหัสนักศึกษา ให้คงไว้เพียงทูเพิลเดียว จึงได้ผลลัพธ์ R ∪ S ที่ประกอบด้วยทูเพิล 4 ทูเพิลดังแสดง

6.2.2.2 อินเตอร์เซกชัน (Intersection)

อินเตอร์เซกชัน คือ การรวมรีเลชันเข้าด้วยกันโดยทูเพิลที่อยู่ในรีเลชันผลลัพธ์ทั้งหมดจะต้องปรากฏอยู่ในรีเลชันที่ถูกดำเนินการทุกรีเลชัน และรีเลชันที่อินเตอร์เซกกันต้องเข้ากันได้ (union-compatible)



รูปแบบกระทำกับรีเลชัน

Relation 1 ∩ Relation 2

ตารางที่ 1 INTERSECT ตารางที่ 2

สัญลักษณ์การดำเนินการอินเตอร์เซกชัน (บน) และ รูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

∩ คือตัวดำเนินการ intersection สำหรับ R และ S หรือ Relation 1 และ Relation 2 เป็นรีเลชันที่จะรวมเข้าด้วยกัน ทั้งสองรีเลชันนั้นต้องเข้ากันได้โดยมีจำนวนแอททริบิวต์เท่ากัน ลำดับของแอททริบิวต์ตรงกัน และโดเมนของแต่ละแอททริบิวต์สัมพันธ์กัน ผลลัพธ์ที่ได้คือทูเพิลที่ปรากฏอยู่ในทั้ง R และ S

ตัวอย่างที่ 6.6 Intersection

R และ S เป็นรีเลชันที่มีข้อมูลของนักศึกษา **R ∩ S** แสดงได้ดังนี้

R

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

∪

S

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21

R

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

S

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21

R ∩ S

sid	name	gpa
B5075666	สมชาย	3.44

R มีนักศึกษา 3 คน และ S มีนักศึกษา 2 คน การ intersect กันนั้นให้ระบุทูเพิลที่ทั้ง 2 รีเลชันมีเหมือนกัน ในตัวอย่างมี 1 ทูเพิล ได้แก่ ทูเพิลของสมชายซึ่งเป็นนักศึกษาคนเดียวกับที่ได้ทำเครื่องหมาย ⇨ ไว้สังเกตได้จากข้อมูลที่เหมือนกันโดยเฉพาะรหัสนักศึกษา และให้คงไว้เพียงทูเพิลเดียว จึงได้ผลลัพธ์ R ∩ S ที่ประกอบ ด้วยทูเพิล 1 ทูเพิลดังแสดง

6.2.2.3 ผลต่างของเซต (Set Difference)

ผลต่างของเซต หรือ **ผลลบ** คือ การสร้างรีเลชันผลลัพธ์ที่ประกอบไปด้วยทูเพิลที่มีอยู่ในรีเลชันของตัวถูกดำเนินการแรก แต่ไม่มีอยู่ในตัวถูกดำเนินการตัวหลัง และรีเลชันที่ถูกกระทำต้องเข้ากันได้ (union-compatible)

รูปแบบกระทำกับรีเลชัน

Relation 1 — Relation 2

ตารางที่ 1 MINUS ตารางที่ 2

สัญลักษณ์การดำเนินการหาผลต่างของเซต (บน) และรูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

— คือตัวดำเนินการ set difference สำหรับ R และ S หรือ Relation 1 และ Relation 2 เป็นรีเลชันที่จะถูกดำเนินการ โดย R เป็นตัวตั้งและ S เป็นตัวลบ ทั้งสองรีเลชันนั้นต้องเข้ากันได้โดยมีจำนวนแอททริบิวต์เท่ากัน ลำดับของแอททริบิวต์ตรงกัน และโดเมนของแต่ละแอททริบิวต์สัมพันธ์กัน ผลลัพธ์ที่ได้คือทูเพิลที่ปรากฏอยู่ในทั้ง R แต่ไม่ปรากฏอยู่ใน S

ตัวอย่างที่ 6.7 Set Difference

R และ S เป็นรีเลชันที่มีข้อมูลของนักศึกษา **R — S** แสดงได้ดังนี้

R

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

—

S

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21

⇨

R

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

S

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21

R — S

sid	name	gpa
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

R มีนักศึกษา 3 คน และ S มีนักศึกษา 2 คน การลบกันนั้นให้ระบุทูเพิลที่มีใน R แต่ไม่ปรากฏใน S จากตัวอย่างพบว่า สมศักดิ์ และ สมน้ำหน้า ปรากฏใน R และไม่มีอยู่ใน S ในขณะที่สมชาย ถึงแม้จะมีปรากฏใน R แต่ว่าปรากฏอยู่ใน S ด้วย ตามที่ระบุด้วยเครื่องหมาย ⇨ สมชายจึงไม่เป็นทูเพิลในผลลัพธ์ ได้ผลลัพธ์ R — S ที่ประกอบ ด้วยทูเพิล 2 ทูเพิลดังแสดง

6.2.2.4 ผลคูณคาร์ทีเซียน (Cartesian Product)

ผลคูณคาร์ทีเซียน คือ การสร้างรีเลชันผลลัพธ์ที่ประกอบไปด้วยทูเพิลเป็นการจับคู่และต่อทูเพิลทุกทูเพิลในรีเลชันของตัวถูกดำเนินการตัวแรก เข้ากับทูเพิลทุกทูเพิลของตัวถูกดำเนินการตัวหลัง

R

a
b

S

1
2
3

R×S

a	1
a	2
a	3
b	1
b	2
b	3

รูปแบบกระทำกับรีเลชัน

Relation 1 × Relation 2

ตารางที่ 1 TIMES ตารางที่ 2

สัญลักษณ์การดำเนินการคูณคาร์ทีเซียนชัน (บน)
และรูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

× คือตัวดำเนินการ Cartesian product สำหรับ R และ S หรือ Relation 1 และ Relation 2 เป็นรีเลชันที่จะถูกดำเนินการ โดย R เป็นตัวตั้งและ S เป็นตัวคูณ ทั้งสองรีเลชันนั้นไม่จำเป็นต้องเข้ากันได้ ผลลัพธ์ที่ได้คือการต่อทูเพิลทั้งหมดใน R ด้วย ทูเพิลทั้งหมดใน S

ตัวอย่างที่ 6.8 Cartesian Product

แสดงข้อมูลการจับคู่การให้คำปรึกษาที่เป็นไปได้ทั้งหมดระหว่างอาจารย์และนักศึกษา โดย R เป็นรีเลชันข้อมูลอาจารย์ซึ่งมีเพียงคอลัมน์เดียว และ S เป็นรีเลชันที่มีข้อมูลของนักศึกษา R × S แสดงได้ดังนี้

R

fname
สถิตย์
พอลล่า

S

sid	name	gpa
B5075666	สมชาย	3.44
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

R × S

fname	sid	name	gpa
สถิตย์	B5075666	สมชาย	3.44
สถิตย์	B5075831	สมศักดิ์	1.80
สถิตย์	B5075832	สมน้ำหน้า	2.00
พอลล่า	B5075666	สมชาย	3.44
พอลล่า	B5075831	สมศักดิ์	1.80
พอลล่า	B5075832	สมน้ำหน้า	2.00

R มีอาจารย์ 2 คน และ S มีนักศึกษา 3 คน การคูณกันนั้นการต่อทูเพิลทั้งหมดใน R ด้วย ทูเพิลทั้งหมดใน S โดยเริ่มจากทูเพิลแรกในรีเลชัน R ได้แก่อาจารย์สถิตย์ ต่อท้ายด้วยข้อมูลนักศึกษาแต่ละคนได้ 3 ทูเพิลในผลลัพธ์ R × S เมื่อทูเพิลแรกในรีเลชัน R ถูกต่อท้ายด้วยทุกทูเพิลในรีเลชัน S ดังนี้แล้ว ต่อไปเป็นการดำเนินการกับทูเพิลถัดมาได้แก่ทูเพิลอาจารย์พอลล่า นำมาต่อท้ายด้วยทูเพิลนักศึกษาใน S เช่นเดียวกันได้อีก 3 ทูเพิล รวมเป็น 6 ทูเพิลในผลลัพธ์ R × S

6.2.3 (การดำเนินการเชื่อม) Join Operations

โดยปกติแล้วเราต้องการผลลัพธ์จากการเชื่อมตารางเข้าด้วยกันแบบ Cartesian product ที่เป็นไปตามเงื่อนไขเท่านั้น Cartesian product สร้างผลลัพธ์ที่มากเกินความจำเป็นและใช้เวลามาก เราจึงใช้การดำเนินการ join แทน การ join เป็นการสร้างรีเลชันที่เกิดจากการรวมรีเลชันเข้าด้วยกันซึ่งสำคัญเป็นอย่างมากในพีชคณิตเชิงสัมพันธ์ การ join เทียบได้กับการสร้าง Cartesian product จากนั้นทำการใช้การดำเนินการ selection เพื่อเลือกเฉพาะทูเพิลที่ตรงตามเงื่อนไข

R

A	B
a	1
b	2

S

B	C
1	x
1	y
3	z

R⋈S

A	B	C
a	1	x
a	2	y

รูปแบบกระทำกับรีเลชัน

Relation 1 ⋈ Relation 2

ตารางที่ 1 JOIN ตารางที่ 2

สัญลักษณ์การดำเนินการ Join (บน)
และรูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

การ join นั้นมีหลายประเภทได้แก่ Theta join, equijoin, natural join, outer join, semijoin

6.2.3.1 Theta join (θ-join)

Theta join มีรูปแบบเป็น $R \bowtie_F S$ เป็นการสร้างรีเลชันที่เป็นไปตามเงื่อนไขใน predicate F ที่เกิดจาก Cartesian ของ R และ S ซึ่ง F คือเงื่อนไขในลักษณะ R.a θ S.b โดยที่ a และ b เป็นแอททริบิวต์ใน R และ S ตามลำดับ และ θ คือตัวดำเนินการเปรียบเทียบ ได้แก่ <, ≤, >, ≥, = และ ≠

Theta join จึงมีรูปแบบที่สามารถเขียนได้อีกรูปแบบหนึ่งคือ $R \bowtie_F S = \sigma_F (R \times S)$

6.2.3.2 Equijoin

Equijoin คือ Theta join รูปแบบหนึ่ง ในกรณีที่ θ เป็นการเปรียบเทียบโดยเครื่องหมาย "=" เท่านั้น เราเรียกการ join ประเภทนี้ว่า **equijoin** ซึ่งเป็นการ join โดยทั่วไปของการใช้งานฐานข้อมูลเชิงสัมพันธ์

ตัวอย่างที่ 6.9 Theta Join/Equijoin

แสดงข้อมูลนักศึกษาและผลการเรียนสำหรับนักศึกษาที่ได้ลงทะเบียนเรียนในรายวิชาต่างๆ

Student $\bowtie_{\text{Student.sid = Enrolled.sid}}$ Enrolled

Student

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

Enrolled

cid	grade	sid
204203	A	**B5075650**
204204	B	**B5075831**
204205	D	**B5075688**
204329	F	**B5075666**
204204	A	**B5075688**

Student $\bowtie_{\text{Student.sid = Enrolled.sid}}$ Enrolled

Student.sid	name	gpa	cid	grade	Enrolled.sid
B5075666	สมชาย	3.44	204329	F	B5075666
B5075688	สมศรี	3.21	204205	D	B5075688
B5075688	สมศรี	3.21	204024	A	B5075688
B5075650	สมศรี	3.82	204203	A	B5075650
B5075831	สมศักดิ์	1.80	204204	B	B5075831

จากตัวอย่าง นอกจากการมองในลักษณะการ selection ที่ Student.sid = Enrolled.sid ของผลคูณ Student × Enrolled แล้ว การดำเนินการ join สามารถมองได้ว่ามีการดำเนินการดังนี้ โดยเริ่มดำเนินการที่ทูเพิลแรกของตาราง Student ได้แก่นักศึกษารหัส B5075666 จากนั้นมาพิจารณาทูเพิลในตาราง Enrolled ว่าทูเพิลใดที่มี sid มีค่าเท่ากับ B5075666 พบว่ามี 1 ทูเพิลได้แก่รายวิชา 204329 ทำให้ได้ผลลัพธ์เป็นทูเพิลแรกใน Student $\bowtie_{\text{Student.sid = Enrolled.sid}}$ Enrolled พิจารณาไม่มีทูเพิลใดใน Enrolled ที่ sid มีค่าเท่ากับ B5075666 อีกแล้ว จึงทำการดำเนินการ join ต่อกับทูเพิลถัดมาของตาราง Student ทูเพิลถัดมา ได้แก่นักศึกษาที่มีรหัส B5075688 ซึ่งลงทะเบียนไว้ 2 รายวิชาคือ 204025 และ 204204 ทำให้ได้เป็นผลลัพธ์เป็นทูเพิลในลำดับที่ 2 และ 3 ใน Student $\bowtie_{\text{Student.sid = Enrolled.sid}}$ Enrolled ดำเนินการเช่นนี้เรื่อยไป ในที่นี้ไม่ปรากฏว่าสมน้ำหน้า รหัส B5075832 ลงทะเบียนรายวิชาใด จึงไม่ปรากฏในตารางผลลัพธ์

6.2.3.3 Natural join

Natural join คือ equijoin แบบหนึ่ง มีรูปแบบเป็น $R \bowtie S$ ซึ่งกระทำบนแอททริบิวต์ที่เหมือนกันในรีเลชัน R และ S แอททริบิวต์ที่เหมือนกันจะคงไว้เพียง 1 แอททริบิวต์

Natural join เป็นการดำเนินการ equijoin กับรีเลชันที่ถูกกระทำโดย join ด้วยแอททริบิวต์ที่มีชื่อเหมือนกันในทั้งสองรีเลชัน โดยแอททริบิวต์ที่เป็นตัวเชื่อมจะคงไว้เพียงแอททริบิวต์เดียว

ตัวอย่างที่ 6.10 Natural Join

จากตัวอย่าง equijoin เป็นการ join โดยใช้แอททริบิวต์ sid จากทั้ง 2 รีเลชันอยู่แล้ว ดังนั้นเราสามารถเขียน equijion ให้อยู่ในรูปของ natural join และแสดงผลลัพธ์ของการดำเนินการได้ดังนี้

Student $\bowtie_{\text{Student.sid = Enrolled.sid}}$ Enrolled

Student.sid	name	gpa	cid	grade	Enrolled.sid
B5075666	สมชาย	3.44	204329	F	B5075666
B5075688	สมศรี	3.21	204205	D	B5075688
B5075688	สมศรี	3.21	204024	A	B5075688
B5075650	สมศรี	3.82	204203	A	B5075650
B5075831	สมศักดิ์	1.80	204204	B	B5075831

Student $\bowtie$ Enrolled

sid	name	gpa	cid	grade
B5075666	สมชาย	3.44	204329	F
B5075688	สมศรี	3.21	204205	D
B5075688	สมศรี	3.21	204024	A
B5075650	สมศรี	3.82	204203	A
B5075831	สมศักดิ์	1.80	204204	B

เนื่องจาก Student $\bowtie_{\text{Student.sid = Enrolled.sid}}$ Enrolled เป็นการ join โดยแอททริบิวต์ sid ทั้งคู่ จึงสามารถตัด predicate ทิ้งได้ เกิดเป็นการ join แบบ natural join ทั้งนี้แอททริบิวต์ Student.sid และ Enrolled.sid เป็นแอททริบิวต์ที่เหมือนกันและใช้ในการ join จึงคงไว้เพียง 1 แอททริบิวต์ ได้ผลลัพธ์เป็นรีเลชัน Student $\bowtie$ Enrolled ในรูปทางขวามือ

6.2.3.4 Outer join

Outer join มีรูปแบบเป็น $R \times S$ (left outer join) คือการ join ระหว่างรีเลชัน R และ S โดยที่ทูเพิลใน R ที่มีค่าของแอททริบิวต์ร่วมที่ใช้ในการ join ไม่สามารถจับคู่กับแอททริบิวต์ร่วมใน S ยังคงปรากฏอยู่ในผลลัพธ์ สำหรับค่าที่ขาดไปในตาราง S กำหนดให้เป็นค่า null

ตัวอย่างที่ 6.11 Outer Join

จากตัวอย่าง natural join สามารถดำเนินการแบบ outer join ได้ดังนี้ **Student $\bowtie$ Enrolled**

Student

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

Enrolled

cid	grade	sid
204203	A	**B5075650**
204204	B	**B5075831**
204205	D	**B5075688**
204329	F	**B5075666**
204204	A	**B5075688**

Student $\bowtie$ Enrolled

sid	name	gpa	cid	grade
B5075666	สมชาย	3.44	204329	F
B5075688	สมศรี	3.21	204205	D
B5075688	สมศรี	3.21	204024	A
B5075650	สมศรี	3.82	204203	A
B5075831	สมศักดิ์	1.80	204204	B
B5075832	สมน้ำหน้า	2.00	null	null

จากตัวอย่างพบว่า การดำเนินการ natural join ระหว่าง Student และ Enrolled จะไม่ปรากฏทูเพิลนักศึกษาที่ชื่อสมน้ำหน้าในผลลัพธ์เนื่องจากไม่สามารถจับคู่รหัสนักศึกษา (sid) ได้ในตาราง Enrolled ได้ แต่

หากกระทำ outer join ทูเพิลของนักศึกษาสมน้ำหน้าจะยังคงปรากฏอยู่ในตารางผลลัพธ์ดังแสดง และค่าที่ว่างอยู่ในตารางจะมีค่าเป็นค่าว่างหรือ null

ตัวอย่างที่ได้กล่าวถึงคือการดำเนินการ left natural outer join คือการทำ natural join (ไม่ได้ระบุ predicate สำหรับการ join) ที่คงค่าทูเพิลของตัวถูกดำเนินการทางซ้ายไว้ทั้งหมด นอกจาก left outer join แล้วเรายังสามารถดำเนินการ right outer join ได้ กล่าวคือเป็นการดำเนินการ outer join ที่คงค่าทูเพิลของตัวถูกดำเนินการทางขวาไว้ทั้งหมดแม้จะไม่มีค่าที่จับคู่กันได้สำหรับแอททริบิวต์ร่วมกับตัวถูกดำเนินการทางซ้าย รวมทั้งการดำเนินการแบบ full outer join ก็สามารถกระทำได้ โดยผลลัพธ์แสดงทูเพิลจากการ join กันของรีเลชันรวมกับทูเพิลของตัวถูกดำเนินการทั้งทางซ้ายและขวาของตัวดำเนินการที่ค่าในแอททริบิวต์ร่วมในการ join ไม่สามารถจับคู่กันได้

6.2.3.5 Semijjoin

Semijoin มีรูปแบบเป็น $\boldsymbol{R} \rhd_F \boldsymbol{S}$ คือการสร้างรีเลชันที่ประกอบไปด้วยทูเพิลของ R ที่ปรากฏในการ join ระหว่างรีเลชัน R และ S

Semijoin มีรูปแบบที่สามารถเขียนได้อีกรูปแบบหนึ่งคือ $R \rhd_F S = \Pi_A (R \times S)$

โดยที่ A คือเซตของแอททริบิวต์ทั้งหมดใน R

Semijoin เป็นการดำเนินการ join ระหว่างรีเลชันและแสดงผลลัพธ์เฉพาะแอททริบิวต์ที่อยู่ในรีเลชันที่เป็นตัวถูกดำเนินการตัวหน้าเท่านั้น การดำเนินการ semijoin นี้มีข้อดีในระบบจัดการฐานข้อมูลเชิงสัมพันธ์ตรงที่เราสามารถลดจำนวนแอททริบิวต์ที่เกี่ยวข้องการดำเนินการ join ทำให้การดำเนินการมีความรวดเร็วขึ้น

ตัวอย่างที่ 6.12 Semijoin

จากตัวอย่าง equijion ให้แสดงข้อมูลนักศึกษาที่ได้ทำการลงทะเบียนเรียนแล้ว

Student $\rhd_{\text{Student.sid = Enrolled.sid}}$ Enrolled

Student $\bowtie_{\text{Student.sid = Enrolled.sid}}$ Enrolled

Student.sid	name	gpa	cid	grade	Enrolled.sid
B5075666	สมชาย	3.44	204329	F	B5075666
B5075688	สมศรี	3.21	204205	D	B5075688
B5075688	สมศรี	3.21	204024	A	B5075688
B5075650	สมศรี	3.82	204203	A	B5075650
B5075831	สมศักดิ์	1.80	204204	B	B5075831

Student $\rhd_{\text{Student.sid = Enrolled.sid}}$ Enrolled

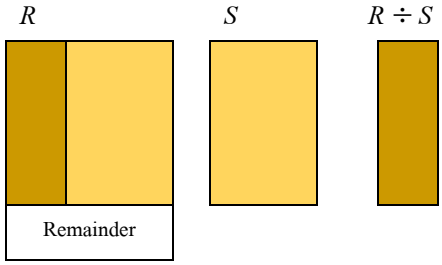
Student.sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075831	สมศักดิ์	1.80

การแสดงข้อมูลตามตัวอย่างตรงกับการ join กันระหว่างรีเลชัน Student และ Enrolled แต่ผลลัพธ์ของ semijoin นั้นแสดงเฉพาะแอททริบิวต์ใน Student ดังแสดงในรูปทางด้านขวา

ในกรณีนี้สามารถเขียนในรูป Student $\rhd$ Enrolled ก็ได้เนื่องจากเป็น natural semijoin

6.2.4 การดำเนินการหาร (Division Operations)

การหาร คือ การสร้างรีเลชันจากสองรีเลชัน โดยที่รีเลชันทั้งสองมีแอททริบิวต์อย่างน้อยหนึ่งแอททริบิวต์ที่เหมือนกัน ผลลัพธ์ที่ได้ จะเป็นค่าของแอททริบิวต์จากรีเลชันที่มีจำนวนแอททริบิวต์มากกว่า ซึ่งเป็นค่าของแอททริบิวต์หนึ่งที่มีค่าหนึ่งที่จับคู่ตรงกับทุกค่าของแอททริบิวต์ที่เหมือนกันนี้ในอีกรีเลชันที่มีแอททริบิวต์น้อยกว่า



รูปแบบกระทำกับรีเลชัน

Relation 1 ÷ Relation 2

ตารางที่ 1 DIVIDEBY ตารางที่ 2

สัญลักษณ์การดำเนินการหารของเซต (บน) และรูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

÷ คือตัวดำเนินการ division สำหรับ R และ S หรือ Relation 1 และ Relation 2 เป็นรีเลชันที่จะถูกดำเนินการ โดย R เป็นตัวตั้งและ S เป็นตัวหาร ทั้งสองรีเลชันนั้นไม่จำเป็นต้องเข้ากันได้ ผลลัพธ์ที่ได้คือการต่อทูเพิลทั้งหมดใน R ด้วย ทูเพิลทั้งหมดใน S

ตัวอย่างที่ 6.13 Division

รีเลชัน V และ W ใช้เป็นตัวอย่างสำหรับ division ซึ่งแสดงไว้ 3 ตัวอย่าง ดังนี้

V

A	B
a	1
a	2
a	3
a	6
b	3
b	4
c	3
d	2
d	4
d	5

W1

B
2

V1 ÷ W1

A
a
d

V

A	B
a	1
a	2
a	3
a	6
b	3
b	4
c	3
d	2
d	4
d	5

W2

B
2
4

V ÷ W2

A
d

V

A	B
a	1
a	2
a	3
a	6
b	3
b	4
c	3
d	2
d	4
d	5

W3

B
1
2
3
6

V ÷ W3

A
a

ในตัวอย่างซ้ายสุด V เป็นตัวตั้งและ W1 เป็นตัวหาร ทูเพิลใดบ้างที่แอททริบิวต์ B ใน V ที่มีค่าตรงกับใน W1 ซึ่งในที่นี้คือค่า 2 ผลปรากฏว่ามีอยู่ 2 ทูเพิล ผลลัพธ์ V ÷ W1 จำได้แก่ a และ d ซึ่งเป็นค่าแอททริบิวต์ในรีเลชันที่มีจำนวนแอททริบิวต์มากกว่าและเป็นตัวตั้ง

ในตัวอย่างกลางนั้น W2 ที่เป็นตัวหารมี 2 ทูเพิลได้แก่ค่า 2 และ 4 ซึ่งทูเพิลที่มีค่าแอททริบิวต์ B เท่ากับ 2 หรือ 4 ในรีเลชัน V นั้นมีอยู่ 4 ทูเพิลที่ได้แรงเงาไว้ อย่างไรก็ตามต้องพิจารณาว่าค่าในแอททริบิวต์ที่เป็นตัวตั้งต้องมีค่าเหมือนกันด้วย จึงมีเพียงทูเพิลที่มีค่าแอททริบิวต์ A ที่มีค่าเป็น d เท่านั้นที่เหมือนกันสำหรับทูเพิลที่แรงเงาไว้ จำเป็นคำตอบของ V ÷ W2 เช่นเดียวกับในตัวอย่างทางขวาสุดที่ทูเพิลใน V ที่มีค่าแอททริบิวต์ B ที่ 1, 2, 3 หรือ 6 สัมพันธ์กับตัวหาร W3 แต่มีเพียงทูเพิลที่มีค่าแอททริบิวต์ A เป็น a เท่านั้นที่มีค่าจับคู่ได้ทั้ง 4 ทูเพิล

ตัวอย่างที่ 6.14 Division

การดำเนินการหารรีเลชันนั้นมักนำมาใช้งานในรูปแบบของข้อคำถามเกี่ยวกับกิจกรรมที่เอนทีตีหลายเอนทิตีกระทำเหมือนกันเช่นการแสดงรายวิชาที่นักศึกษากลุ่มหนึ่งลงทะเบียนเรียนเหมือนกันทั้งหมด ดังตัวอย่างต่อไปนี้

Enrolled1

cid	*sid*
204203	B5075650
204204	B5075831
204205	B5075688
204329	B5075666
204204	B5075688
204204	B5075666

Student1

sid
B5075688
B5075666

Enrolled1

cid	*sid*
204203	B5075650
204204	B5075831
204205	B5075688
204329	B5075666
⇨ **204204**	B5075688
⇨ **204204**	B5075666

Enrolled1 ÷ Student1

cid
204204

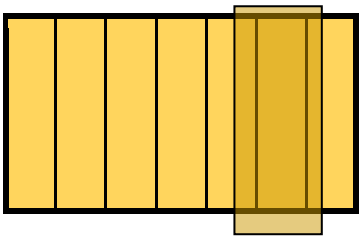
ทูเพิลที่มีแอททริบิวต์ตรงกันกับตัวหาร (Student1) มี 4 ทูเพิล ดังที่ได้แรเงาไว้ เป็นการแสดงว่านักศึกษา B5075688 และ B5075666 ลงทะเบียนรายวิชาใดบ้าง แต่เราต้องการทราบเพียงรายวิชาเรียนที่เหมือนกันซึ่งเป็นผลหารของรีเลชัน พบว่ามีเพียงรายวิชา 204204 เท่านั้นที่ทั้ง 2 คนลงทะเบียนเรียนทั้งคู่ซึ่งได้ทำเครื่องหมาย ⇨ ระบุไว้ ผลลัพธ์ที่ได้เป็นแอททริบิวต์ที่เหมือนกันนี้ในรีเลชันตัวตั้ง (Enrolled) ได้ผลลัพธ์ Enrolled1 ÷ Student1 ดังแสดง

6.2.5 การดำเนินการแบบเป็นชุดและจัดกลุ่ม (Aggregation and Grouping Operations)

บ่อยครั้งที่เราต้องการดำเนินการกับข้อมูลในรูปแบบของการคำนวณใดๆ ในกลุ่มของข้อมูล (aggregation) เช่นการหาผลรวมยอดขายสินค้า และการดำเนินการนั้นยังอาจมีการจัดกลุ่มของข้อมูล (grouping) เช่นการหาผลรวมแยกตามสาขา ทั้งนี้พีชคณิตเชิงสัมพันธ์พื้นฐานไม่สามารถกระทำได้ ในหัวข้อนี้อธิบายการดำเนินการเพิ่มเติมของพีชคณิตเชิงสัมพันธ์ที่สามารถจัดการข้อมูลเป็นชุดๆ ได้

6.2.5.1 Aggregate Operations

Aggregate operation คือ การกระทำบนรีเลชันโดยการใช้ฟังก์ชันแบบ aggregate function กับแอททริบิวต์ที่กำหนด



รูปแบบกระทำกับรีเลชัน

$\mathfrak{I}_{AL}$(Relation)

ชื่อตาราง AL

สัญลักษณ์การดำเนินการโปรเจคชัน (บน) และรูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

$\mathfrak{I}$ คือตัวดำเนินการ aggregate operation และ AL คือรายการฟังก์ชันที่กำหนดให้กระทำกับรีเลชัน รายการรีเลชันแต่ละรายการประกอบด้วยฟังก์ชันและแอททริบิวต์ที่กำหนดสำหรับฟังก์ชันนั้นๆ aggregate function หลักๆ ได้แก่

COUNT—นับจำนวนของค่าในแอททริบิวต์ที่กำหนด

SUM—หาผลรวมของค่าในแอททริบิวต์ที่กำหนด

AVG—หาค่าเฉลี่ยของค่าในแอททริบิวต์ที่กำหนด

MIN/MAX—หาค่าต่ำสุด/สูงสุด

ตัวอย่างที่ 6.15 Aggregate Operations

แสดงจำนวนนักศึกษาโดยการนับด้วยรหัสนักศึกษา พร้อมทั้งแสดงเกรดเฉลี่ย และอายุเฉลี่ยของนักศึกษา

$$\rho_{sReports}\ (sCount, maxAge, avgGpa)\ \mathfrak{I}_{COUNT\ sid,\ MAX\ age,\ AVG\ gpa}\ (Student)$$

Student

COUNT MAX AVG

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

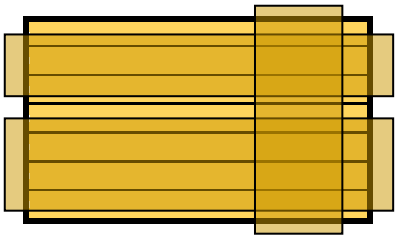
sReports

sCount	maxAge	avgGpa
5	22	2.85

$\rho_{sReports}$ **(sCount, maxAge, avgGpa) คือการตั้งชื่อของผลลัพธ์**ที่ได้ในที่นี้ระบุชื่อรีเลชันเป็น sReports กำหนดแอททริบิวต์ให้มีชื่อว่า sCount, maxAge และ avgGpa ตามลำดับ $\mathfrak{I}$ เป็นเครื่องหมายการดำเนินการแบบ aggregate และรายการ COUNT sid, MAX age, AVG gpa คือ aggregate list ซึ่งแต่ละรายการประกอบไปด้วย aggregate function และแอททริบิวต์ที่ฟังก์ชัน นั้นๆ กระทำ ซึ่งรายการแรกได้แก่ฟังก์ชัน COUNT ซึ่งเป็นการนับจำนวนรหัสนักศึกษา (sid) รายการถัดมาได้แก่ฟังก์ชัน MAX ซึ่งเป็นการหาอายุ (age) ที่มากที่สุดของนักศึกษา และรายการสุดท้ายได้แก่ AVG ซึ่งเป็นฟังก์ชันหาค่าเฉลี่ยของเกรดเฉลี่ย (gpa) ของนักศึกษาทั้งหมดในรีเลชัน Student

6.2.5.2 Grouping Operations

Grouping operation คือ การกระทำบนรีเลชันโดยการใช้ฟังก์ชันแบบ aggregate function กับแอททริบิวต์ที่กำหนด และจัดกลุ่มตามแอททริบิวต์ที่ระบุ



รูปแบบกระทำกับรีเลชัน

$_{GA}\Im_{AL}$(Relation)

ชื่อตาราง AL GA

สัญลักษณ์การดำเนินการหาผลต่างของเซต (บน) และรูปแบบแบบคำสั่ง (ล่าง) ที่สัมพันธ์กัน

$\Im$ คือตัวดำเนินการ aggregate operation และ AL คือรายการฟังก์ชันที่กำหนดให้กระทำกับรีเลชัน รายการรีเลชันแต่ละรายการประกอบด้วยฟังก์ชันและแอททริบิวต์ที่กำหนดสำหรับฟังก์ชันนั้นๆ เช่นเดียวกันกับ aggregate operation ในหัวข้อที่แล้ว แต่การดำเนินการนั้นกระทำเป็นกลุ่มๆ สำหรับทูเพิลที่มีค่าของแอททริบิวต์ที่ระบุไว้ใน grouping attributes (GA) ที่เหมือนกัน

ตัวอย่างที่ 6.16 Grouping Operations

แสดงจำนวนนักศึกษาโดยการนับด้วยรหัสนักศึกษา พร้อมทั้งแสดงเกรดเฉลี่ย และอายุเฉลี่ยของนักศึกษา

$$\rho_{sReports}\ (age, avgGpa)_{age}\Im_{AVG\ gpa}\ (Student)$$

GROUP BY AVG

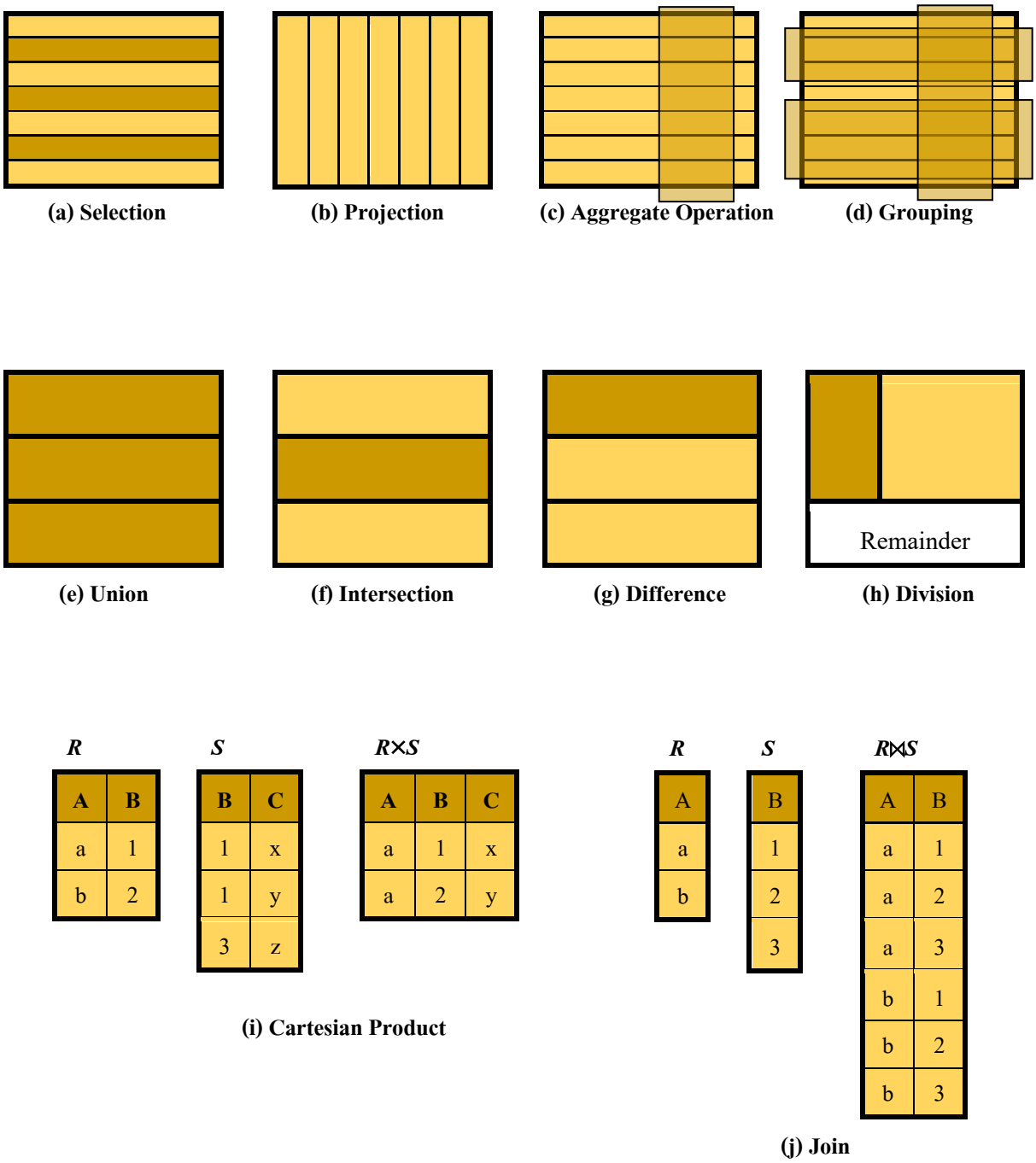
Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

sReports

age	avgGpa
18	3.33
19	3.82
21	1.80
22	2.00

$\rho_{sReports}$ (age, avgGpa) คือการตั้งชื่อของผลลัพธ์ที่ได้ในที่นี้ระบุชื่อรีเลชันเป็น sReports กำหนดแอททริบิวต์ให้มีชื่อว่า age และ avgGpa ตามลำดับ $\Im$ เป็นเครื่องหมายการดำเนินการแบบ aggregate และรายการ AVG gpa คือ aggregate list ซึ่งมีเพียงรายการเดียวได้แก่ AVG ซึ่งเป็นฟังก์ชันหาค่าเฉลี่ยของเกรดเฉลี่ย (gpa) ของนักศึกษาจำแนกตามอายุ (age) ซึ่งระบุไว้ในตำแหน่งของ GA หรือหน้าเครื่องหมาย $\Im$ ในตัวอย่างสามารถจัดกลุ่มนักศึกษาที่มีอายุเท่ากันได้เป็น 4 กลุ่ม จากนั้นหาค่าเฉลี่ยของเกรดเฉลี่ยของนักศึกษาในแต่ละกลุ่มได้เป็นผลลัพธ์ sReports ดังแสดง



ภาพที่ 6-1 (a)-(j) สรุปรูปแบบการดำเนินการของแคลคูลัสเชิงสัมพันธ์

6.3 แคลคูลัสเชิงสัมพันธ์ (Relational Calculus)

แคลคูลัสเชิงสัมพันธ์เป็นการดำเนินการบนโมเดลเชิงสัมพันธ์รูปแบบหนึ่ง ซึ่งต่างจากพีชคณิตเชิงสัมพันธ์ตรงที่แคลคูลัสเชิงสัมพันธ์นี้ เป็นการดำเนินการที่ไม่ได้ระบุขั้นตอนการได้มาซึ่งผลลัพธ์ หรือไม่ได้ระบุว่า ทำอย่างไร (how) เหมือนในพีชคณิตเชิงสัมพันธ์ แต่เป็นการระบุว่าอะไร (what) เป็นผลลัพธ์ที่ต้องการจากความสัมพันธ์ต่างๆ ในโมเดลเชิงสัมพันธ์ที่ระบุไว้ในข้อคำถามของแคลคูลัสเชิงสัมพันธ์ แคลคูลัสเชิงสัมพันธ์ไม่ได้เกี่ยวข้องกับ differential calculus หรือ integral calculus แต่ได้ชื่อมาจากศาสตร์ที่ใช้สัญลักษณ์ทางตรรกะที่เรียกว่า predicate calculus ซึ่งแคลคูลัสเชิงสัมพันธ์นี้ใช้สำหรับยืนยันว่าภาษาสำหรับแบบจำลองเชิงสัมพันธ์ใดๆ ที่สร้างขึ้นจะสมบูรณ์หรือเรียกว่าบริบูรณ์เชิงสัมพันธ์ (relationally complete) ก็ต่อเมื่อมี power หรือสร้างผลลัพธ์ได้เช่นเดียวกันกับการแทนข้อคำถามด้วยแคลคูลัสเชิงสัมพันธ์ ในโมเดลเชิงสัมพันธ์แคลคูลัสเชิงสัมพันธ์แบ่งเป็น 2 รูปแบบได้แก่ **แคลคูลัสเชิงสัมพันธ์แบบทูเพิล (tuple relational calculus)** และ **แคลคูลัสเชิงสัมพันธ์แบบโดเมน (domain relational calculus)**

Predicate ใน predicate calculus นั้นคือฟังก์ชันในการหาค่าความจริง และค่าอาร์กิวเมนต์ที่จะแทนลงในฟังก์ชันเพื่อหาค่าความจริง เมื่อเราทำการแทนอาร์กิวเมนต์ลงในฟังก์ชัน เราจะได้ประพจน์ ซึ่งหมายถึงข้อความที่สามารถหาค่าความจริงได้ เช่น “X > 1” เป็นฟังก์ชัน เมื่อแทนค่า X ด้วย 1 จะได้ประพจน์ “1 > 1” ซึ่งมีค่าความจริงเป็นเท็จ

ถ้า P เป็น predicate แล้วเราสามารถเขียนเซตของค่า x ทั้งหมดที่ทำให้ P เป็นจริงสำหรับค่า x ด้วย:

$\{x \mid P(x)\}$

เราสามารถเชื่อม predicates เข้าด้วยกันได้ด้วยตัวดำเนินการทางตรรกะ $\wedge$ (AND), $\vee$ (OR), ~ (NOT) เพื่อสร้าง predicate ประกอบกันได้ เป็นสูตรในที่สุด

6.3.1 แคลคูลัสเชิงสัมพันธ์แบบทูเพิล (Tuple Relational Calculus)

สำหรับแคลคูลัสเชิงสัมพันธ์แบบทูเพิลนั้นสามารถมองได้ง่ายๆ คือการเขียนเซต $\{x \mid P(x)\}$ โดยในที่นี้ x คือทูเพิล และ P(x) คือ predicate ที่เป็นฟังก์ชันหาค่าความจริงเกี่ยวกับทูเพิล x ทั้งนี้ P เป็นตัวกำหนดโดเมนของทูเพิล เช่น Student(x) หมายความว่า x เป็นทูเพิลในตารางความสัมพันธ์ Student ทำให้เราสามารถใช้แคลคูลัสเชิงสัมพันธ์แบบทูเพิลเพื่อแทนข้อคำถามอย่างง่ายๆ ได้ดังนี้

“แสดงนักศึกษาที่มีรหัสนักศึกษา B3800546”

สามารถเขียนด้วยแคลคูลัสเชิงสัมพันธ์แบบทูเพิลดังนี้

{ S | Student(S) ∧ S.sid >= ‘B3800546’ }

พิจารณาเริ่มต้นที่ Student(S) ซึ่ง S คือทูเพิลทั้งหมดในตารางความสัมพันธ์ Student จากนั้นพิจารณา S.sid >= ‘B3800546’ ซึ่งหมาความว่าทูเพิล S ใดๆ ที่แอททริบิวต์ sid ของทูเพิลนั้นมีค่าเท่ากับ B3800546 ซึ่งในที่นี้ทูเพิล S ถูกกำหนดให้เป็นทูเพิลของตารางนักศึกษาแล้วจาก predicate ก่อนหน้า สูตร **Student(S) ∧ S.sid >= ‘B3800546’** จึงเป็นฟังก์ชันของ S ที่จะเป็นจริงเมื่อ S นั้นเป็นทูเพิลของนักศึกษา B3800546 นั่นเอง สำหรับทูเพิล S ที่ทำให้สูตรนี้เป็นจริง S ที่ขีดเส้นใต้ใน **{ <u>S</u> | Student(S) ∧ S.sid >= ‘B3800546’ }** จึงเป็น

รายการทูเพิลทั้งหมดที่ทำให้สูตรเป็นจริง ซึ่งเป็นหลักการเขียนของ predicate calculus และเซต การเขียนเซตดังกล่าวจึงเป็นการแทนข้อคำถาม **"แสดงนักศึกษาที่มีรหัสนักศึกษา B3800546"** นั่นเอง

ซึ่งข้อคำถามนี้สามารถเขียนได้ด้วยพีชคณิตเชิงสัมพันธ์ดังนี้

$\sigma_{\mathbf{sid = 'B3800546'}}$ **(Student)**

เมื่อนำมาเปรียบเทียบกับ **{ S | Student(S) ∧ S.sid >= 'B3800546' }** แล้ว พบว่าข้อแตกต่างกันนั่นคือในพีชคณิตเชิงสัมพันธ์จะต้องใช้ตัวดำเนินการ selection ที่บ่งบอกถึงว่าจะต้องดำเนินการอย่างไรเพื่อให้ได้ระเบียนผลลัพธ์ ต่างจากในแคลคูลัสเชิงสัมพันธ์ที่เป็นการบ่งบอกว่า อะไรเป็นระเบียบผลลัพธ์ ซึ่งในที่นี้คือระเบียนนักศึกษารหัส B3800546 โดยไม่ต้องบอกว่าจะต้องดำเนินการอย่างไรจึงจะได้ระเบียนมา

นอกจากนี้ในแคลคูลัสเชิงสัมพันธ์ยังมีเครื่องหมายตัวบ่งปริมาณ (quantifier) ∃ (existential quantifier) และ ∀ (universal quantifier) เพื่อระบุว่า predicate ที่กำหนดครอบคลุม instance ของทูเพิลมากน้อยเท่าใด โดย ∃ ใช้ในสูตรที่จะต้องเป็นจริงอย่างน้อย 1 instance หรือกล่าวได้ว่า "มีอยู่บ้าง" ในขณะที่ ∀ ใช้ในกรณีที่ instance ทั้งหมดต้องเป็นจริง เช่น (∀S)(S.sid ≠ 'B3800546') ซึ่งได้แก่ทูเพิลของนักศึกษาทั้งหมดที่ไม่ใช่นักศึกษารหัส B3800546 เราสามารถใช้ De Morgan's laws ในการแปลงระหว่างการใช้ ∀ และ ∃ ได้ ซึ่งไม่ได้อธิบายในที่นี้

ต่อไปนี้จะเป็นตัวอย่างเพิ่มเติมของการแทนข้อคำถามด้วยแคลคูลัสเชิงสัมพันธ์แบบทูเพิล

ตัวอย่างที่ 6.17

(ก) "ให้แสดงข้อมูลนักศึกษาทั้งหมดที่เกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป**และ**มีอายุมากกว่า 20 ปี"

{ S | Student(S) ∧ S.gpa >= 2.00 ∧ S.age>20 }

จากตัวอย่าง S เป็น**ตัวแปรทูเพิล (tuple variable)** ซึ่งกำหนดให้อยู่ในความสัมพันธ์ Student ดังนั้น S จึงเป็นทูเพิลทั้งหมดของตาราง Student แต่มีการคัดเฉพาะทูเพิลที่มี gpa และ age ตามที่ระบุ ข้อคำถามนี้สามารถใช้พีชคณิตเชิงสัมพันธ์เขียนแทนได้คือ $\sigma_{\mathbf{gpa >= 2.00 \wedge age > 20}}$ **(Student)** ซึ่งตรงกับตัวอย่างที่ 6.1 นั่นเอง

(ข) "ให้แสดงข้อมูลนักศึกษาทั้งหมดที่เกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป**หรือ**นักศึกษาที่มีอายุมากกว่า 20 ปี"

{ S | Student(S) ∧ (S.gpa >= 2.00 ∨ S.age>20) }

ตัวอย่างนี้สามารถเปรียบเทียบได้กับการแทนข้อคำถามด้วยพีชคณิตเชิงสัมพันธ์ในตัวอย่างที่ 6.2

(ค) "ให้แสดงรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยของนักศึกษาทั้งหมด"

{ S.sid, S.gpa | Student(S) }

เราสามารถระบุคอลัมน์ข้อมูลที่ต้องการได้โดยการกำหนดดังแสดง S.sid และ S.gpa จะปรากฏในผลลัพธ์เท่านั้นในขณะที่คอลัมน์อื่นๆ ไม่ได้แสดง ซึ่งเสมือนการระบุผลลัพธ์ว่าต้องการคอลัมน์ใด แต่ไม่ได้

ระบุว่าต้องดำเนินการ projection เหมือนในพีชคณิตเชิงสัมพันธ์ สามารถเปรียบเทียบได้กับการแทนข้อคำถามด้วย $\Pi_{sid, name, gpa}(Student)$ ในตัวอย่างที่ 6.3

(ง) "ให้แสดงรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยของนักศึกษาที่มีเกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป"

$\{ S.sid, S.name, S.gpa \mid Student(S) \wedge S.gpa >= 2.00 \}$

ตัวอย่างนี้สามารถเปรียบเทียบได้กับการแทนข้อคำถามด้วยพีชคณิตเชิงสัมพันธ์ในตัวอย่างที่ 6.4 ได้แก่ $\sigma_{gpa >= 2.00} (\Pi_{sid, name, gpa} (Student))$ หรือ $\Pi_{sid, name, gpa}(\sigma_{gpa >= 2.00} (Student))$ ซึ่งจากตัวอย่างนี้จะทำให้เห็นภาพชัดเจนถึงจุดแตกต่างที่สำคัญระหว่างแคลคูลัสเชิงสัมพันธ์กับพีชคณิตเชิงสัมพันธ์ นั่นคือในพีชคณิตเชิงสัมพันธ์เราต้องระบุว่าจะดำเนินการด้วยตัวดำเนินการ selection หรือ projection ก่อน แต่ในแคลคูลัสเชิงสัมพันธ์ไม่ได้คำนึงถึง

(จ) "แสดงข้อมูลนักศึกษาและผลการเรียนสำหรับนักศึกษาที่ได้ลงทะเบียนเรียนในรายวิชาต่างๆ"

$Student \bowtie_{Student.sid = Enrolled.sid} Enrolled$

$\{ S \mid Student(S) \wedge (\exists E)(Enrolled(E) \wedge (S.sid = E.sid)) \}$

ในกรณีนี้จะเห็นว่าเราจำเป็นต้องใช้เครื่องหมายบ่งปริมาณ $\exists E$ เพื่อระบุว่ามีอย่างน้อย 1 ทูเพิลที่ทำให้สูตรเป็นจริง โดย E แทนทูเพิลในรีเลชัน Enrolled ดังนั้นทูเพิลใดของ S (Student) และ E (Enrolled) ที่เป็นไปตามเงื่อนไข S.sid = E.sid เป็นสมาชิกของเซตผลลัพธ์ได้ ตัวอย่างนี้สามารถเปรียบเทียบได้กับการแทนข้อคำถามด้วยพีชคณิตเชิงสัมพันธ์ในตัวอย่างที่ 6.9

(ฉ) "แสดงข้อมูลนักศึกษาและผลการเรียนสำหรับนักศึกษาที่ลงทะเบียนเรียนในรายวิชา 204204"

$\{ S.sid, S.name, S.gpa \mid Student(S) \wedge (\exists E)(Enrolled(E) \wedge (S.sid = E.sid) \wedge (E.sid = '204204')) \}$

จากตัวอย่าง สามารถอธิบายได้เช่นเดียวกันกับตัวอย่าง (จ) แต่เพิ่งเงื่อนไข E.sid = '204204' เพื่อแสดงเฉพาะข้อมูลของนักศึกษาและการลงทะเบียนเฉพาะรายวิชา 204204 เท่านั้น ซึ่งจะตรงกับการแทนข้อคำถามด้วยพีชคณิตเชิงสัมพันธ์

$\Pi_{sid, name, gpa} (\sigma_{Enrolled.cid='204204'} (Student \bowtie_{Student.sid = Enrolled.sid} Enrolled))$

6.3.2 แคลคูลัสเชิงสัมพันธ์แบบโดเมน (Domain Relational Calculus)

แคลคูลัสเชิงสัมพันธ์แบบโดเมนนั้นมีแนวคิดเช่นเดียวกับแคลคูลัสเชิงสัมพันธ์แบบทูเพิล แต่ต่างกันตรงที่ในแคลคูลัสเชิงสัมพันธ์แบบโดเมนนี้จะเน้นที่ฟังก์ชันของโดเมน (โดเมนของแอททริบิวต์) กับอาร์กิวเมนต์ที่เป็นโดเมน หรือกล่าวให้เข้าใจง่ายขึ้นก็คือการมองในแง่ของแอททริบิวต์เป็นหลัก (เนื่องจากโดเมนนั้นผูกติดกับแอททริบิวต์นั่นเอง) ซึ่งสามารถยกตัวอย่างการแทนข้อคำถามด้วยแคลคูลัสเชิงสัมพันธ์แบบโดเมนได้ดังนี้

ตัวอย่างที่ 6.18

ในตัวอย่างนี้ ($\exists d_1, \exists d_2, \ldots$) ซึ่งเป็นการแสดงการบ่งปริมาณของโดเมนจะเขียนโดยย่อว่า ($\exists d_1, d_2, \ldots$)

(ก) "ให้แสดงข้อมูลนักศึกษาทั้งหมดที่เกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป**และ**มีอายุมากกว่า 20 ปี"

{ S | Student(S) ∧ S.gpa >= 2.00 ∧ S.age>20 }

{ cid, sid, name, login, age, gpa | (∃age, gpa)(Student(cid, sid, name, login, age, gpa) ∧ age > 20 ∧ gpa >= 2.00) }

จากตัวอย่าง ให้พิจารณา **(∃age, gpa)** เป็นการระบุโดเมนหรือแอทริบิวต์ใดบ้างที่จะทำการกำหนดเงื่อนไข โดย **age** และ **gpa** ในที่นี้เป็นตัวแปรที่กำกับโดย **Student(cid, sid, name, login, age, gpa)** ที่อยู่ติดกันโดยสิ่งที่อยู่ใน Student(…) นั้นเป็นการกำหนดค่า**ตัวแปรโดเมน (domain variable)** โดย cid, sid, name, login, age และ gpa นั่นเป็นตัวแปรแทนโดเมนของแอทริบิวต์ที่มีลำดับสัมพันธ์กันกับลำดับของแอททริบิวต์ในรีเลชัน cid, sid, name, login, age และ gpa สามารถแทนด้วยคำอื่นๆ ก็ได้เนื่องจากเป็นตัวแปรเท่านั้น ไม่จำเป็นต้องเป็นชื่อเดียวกันกับแอททริบิวต์ age และ gpa ใน (∃age, gpa) ถูกกำกับค่าไว้ด้วยตัวแปรใน Student(…) นี้ **Student(cid, sid, name, login, age, gpa)** หมายความว่าแอททริบิวต์ตามที่ระบุของตารางนักศึกษา **age > 20** ระบุว่าสมาชิกในผลลัพธ์มีค่าในโดเมนของ age มากกว่า 20 เท่านั้น ในกรณีนี้ age ก็เป็นตัวแปรตัวเดียวกันกับที่อ้างถึงใน Student(…) ซึ่งจะต้องเป็นแอททริบิวต์ในรีเลชัน Student ท่านั้น เราจึงไม่จำเป็นต้องระบุ S.age อีก เช่นเดียวกันกับ gpa ใน **gpa >= 2.00** การแทนตัวแปรในลักษณะนี้ยังเป็นการยืนยันว่าค่าที่ทำการตรวจสอบค่าความจริงเป็นค่าจากทูเพิลเดียวกัน ตัวอย่างข้างต้นนี้จึงเป็นการแสดงข้อมูลแอททริบิวต์ทั้งหมด **cid, sid, name, login, age, gpa** ที่เป็นไปตามเงื่อนไขคือเป็นนักศึกษาที่มีอายุมากกว่า 20 และเกรดเฉลี่ยมากกว่าหรือเท่ากับ 20 สามารถกล่าวให้เห็นภาพชัดเจนได้ว่าแคลคูลัสเชิงสัมพันธ์แบบโดเมนนี้อาศัยการกำหนดค่าตัวแปรแอททริบิวต์แทนทูเพิลนั่นเอง การสร้างสูตรและ predicate นั้นใช้หลักเช่นเดียวกันกับแคลคูลัสเชิงสัมพันธ์แบบทูเพิล

การแทนข้อคำถามด้วยแคลคูลัสเชิงสัมพันธ์แบบโดเมนตามตัวอย่างนี้ มีค่าเท่ากันกับการแทนด้วยแคลคูลัสเชิงสัมพันธ์แบบทูเพิลในตัวอย่างที่ 6.17 (ก) และพีชคณิตเชิงสัมพันธ์ในตัวอย่างที่ 6.1 และข้อของตัวอย่างของแคลคูลัสเชิงสัมพันธ์แบบโดเมนต่อๆ ไปจะสอดคล้องกันกับแคลคูลัสเชิงสัมพันธ์แบบทูเพิลในตัวอย่างที่ 6.17 และสอดคล้องกับพีชคณิตเชิงสัมพันธ์ที่ระบุไว้ในตัวอย่างที่ 6.17

(ข) "ให้แสดงข้อมูลนักศึกษาทั้งหมดที่เกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป**หรือ**นักศึกษาที่มีอายุมากกว่า 20 ปี"

{ cid, sid, name, login, age, gpa | (∃age, gpa)(Student(cid, sid, name, login, age, gpa) ∧ age > 20 ∨ gpa >= 2.00) }

เปรียบเทียบได้กับตัวอย่างที่ 6.17 (ข)

(ค) "ให้แสดงรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยของนักศึกษาทั้งหมด"

{ sid, gpa | Student(cid, sid, name, login, age, gpa) }

เปรียบเทียบได้กับตัวอย่างที่ 6.17 (ค)

(ง) "ให้แสดงรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยของนักศึกษาที่มีเกรดเฉลี่ยตั้งแต่ 2.00 ขึ้นไป"

{ sid, name, gpa | (∃gpa)(Student(cid, sid, name, login, age, gpa) ∧ gpa >= 2.00) }

เปรียบเทียบได้กับตัวอย่างที่ 6.17 (ง)

(จ) "แสดงข้อมูลนักศึกษาและผลการเรียนสำหรับนักศึกษาที่ได้ลงทะเบียนเรียนในรายวิชาต่างๆ"

{ cid, sid, name, login, age, gpa | (∃sid1)(Student(cid, sid, name, login, age, gpa) ∧ Enrolled(cid, sid1, grade) ∧ (sid = sid1)) }

จากตัวอย่างพบว่ามีการกำหนดชื่อตัวแปรโดเมน sid และ sid1 เพื่อไม่ให้ซ้ำกัน เนื่องจากเป็นการแทนโดเมนของแอททริบิวต์จากคนละตาราง โดย sid ถูกกำหนดเป็นตัวแปรของโดเมน sid ในตาราง Student และ sid1 เป็นตัวแปรของโดเมน sid ในตาราง Enrolled

เปรียบเทียบได้กับตัวอย่างที่ 6.17 (จ)

(ฉ) "แสดงข้อมูลนักศึกษาและผลการเรียนสำหรับนักศึกษาที่ลงทะเบียนเรียนในรายวิชา 204204"

{ sid, name, gpa | (∃sid1, cid1)(Student(cid, sid, name, login, age, gpa) ∧ Enrolled(cid1, sid1, grade) ∧ (sid = sid1) ∧ cid1 = '204204') }

จากตัวอย่างสามารถอธิบายได้เช่นเดียวกันกับตัวอย่าง (จ) สำหรับในแคลคูลัสเชิงสัมพันธ์แบบโดเมน เราสามารถเขียนเซตข้างบนใหม่ได้โดยการแทนตัวแปรโดเมนด้วยค่าเลยก็ได้ จากตัวอย่างหมายความว่าเป็นการกำหนดโดเมนให้เป็น 204204 เท่านั้น

{ sid, name, gpa | (∃sid1, cid1)(Student(cid, sid, name, login, age, gpa) ∧ Enrolled('204204', sid1, grade) ∧ (sid = sid1)) }

เปรียบเทียบได้กับตัวอย่างที่ 6.17 (ฉ)

6.4 บทสรุป

พีชคณิตเชิงสัมพันธ์และแคลคูลัสเชิงสัมพันธ์เป็นพื้นฐานของการจัดการฐานข้อมูลเชิงสัมพันธ์ โดยเฉพาะเป็นพื้นฐานของ SQL ซึ่งเป็นภาษาหลักที่ใช้ในการใช้งานระบบจัดการฐานข้อมูลเชิงสัมพันธ์ พีชคณิตเชิงสัมพันธ์เป็นการแทนข้อคำถามในการแสดงข้อมูลโดยใช้ตัวดำเนินการเพื่อระบุวิธีการที่จะได้มาซึ่งผลลัพธ์ที่ต้องการ ในขณะที่แคลคูลัสเชิงสัมพันธ์ไม่ได้ระบุวิธีการแสดงข้อมูลแต่เป็นการระบุผลลัพธ์ที่ต้องการ พีชคณิตเชิงสัมพันธ์สอดคล้องเป็นอย่างมากกับรูปแบบและการทำงานของ SQL ซึ่งเป็นภาษาเชิงกระบวนคำสั่ง ในขณะที่แคลคูลัสเชิงสัมพันธ์แทนข้อคำถามในลักษณะของภาษาเชิงไรกระบวนคำสั่งได้ และใช้เป็นหลักในการทดสอบ power ของภาษาที่ใช้กับฐานข้อมูลว่าสมบูรณ์หรือที่เรียกว่าบริบูรณ์เชิงสัมพันธ์หรือไม่

การดำเนินการของพีชคณิตเชิงสัมพันธ์ได้แก่ selection เป็นแสดงข้อมูลทูเพิลที่มีค่าของคอลัมน์ตรงกับเงื่อนไขกำหนด projection เป็นการระบุคอลัมน์ที่ต้องการ Cartesian product เป็นการจับคู่ความสัมพันธ์ที่เป็นไปได้ทั้งหมดระหว่างรีเลชัน union เป็นการรวมรีเลชันเข้าด้วยกันโดยตัดทูเพิลที่ซ้ำกันให้เหลือเพียง 1 ทูเพิล intersection เป็นการรวมรีเลชันโดยแสดงผลเฉพาะทูเพิลที่ซ้ำกันระหว่างรีเลชัน set difference เป็นการดำเนินการเพื่อให้ได้ทูเพิลที่มีในรีเลชันหนึ่งแต่ไม่มีในอีกรีเลชันหนึ่ง join เป็นการเชื่อมตารางความสัมพันธ์สำหรับทูเพิลที่มีค่าสัมพันธ์และอ้างอิงถึงกัน division เป็นการหาชุดของค่าในลักษณะที่ซ้ำกับค่าที่ระบุเป็นชุดๆ

แคลคูลัสเชิงสัมพันธ์อาศัย predicate calculus เป็นรูปแบบในการแทนข้อคำถามและรีเลชันผลลัพธ์ในแบบจำลองเชิงสัมพันธ์ แบ่งออกเป็นแคลคูลัสเชิงสัมพันธ์แบบทูเพิล และแคลคูลัสเชิงสัมพันธ์แบบโดเมน แคลคูลัสเชิงสัมพันธ์แทนรีเลชันด้วยสมาชิกของเซตที่ทำให้เงื่อนไขในสูตรของ predicate calculus เป็นจริง แคลคูลัสเชิงสัมพันธ์แบบทูเพิลจะใช้ตัวแปรทูเพิลในการตรวจสอบเงื่อนไขต่างๆ ในขณะที่แคลคูลัสเชิงสัมพันธ์แบบโดเมนจะใช้ตัวแปรโดเมนที่ในแอททริบิวต์ในการตรวจสอบค่าความจริงในสูตร

การสามารถแทนข้อคำถามด้วยทั้งพีชคณิตเชิงสัมพันธ์และแคลคูลัสเชิงสัมพันธ์นี้ จะเป็นพื้นฐานในการสร้างคำสั่ง SQL ที่ใช้ในการพัฒนาระบบฐานข้อมูลต่อไป

6.5 แบบฝึกหัดท้ายบท

1. พีชคณิตเชิงสัมพันธ์ต่างจากแคลคูลัสเชิงสัมพันธ์อย่างไร
2. พิจารณา schema ของระบบสำรองที่พักโรงแรม "จอง-ฉัน-สิ" ระบบจองโรงแรมชื่อดังในประเทศไทย (Begg & Connolly, 2004)

Hotel	(hotel_no, hotel_name, hotel_address)
Room	(room_no, hotel_no, type, price)
Booking	(hotel_no, guest_no, date_from, date_to, room_no)
Guest	(guest_no, guest_name, guest_address)

โดยที่

Hotel บันทึกรายละเอียดของโรงแรม และ hotel_no เป็น primary key

Room บันทึกรายละเอียดห้องของแต่ละโรงแรม (hotel_no, room_no) รวมกันเป็น primary key

Booking บันทึกรายละเอียดของการจองห้องพัก primary key คือ (hotel_no, guest_no, date_from)

Guest บันทึกรายละเอียดของแขกของโรงแรม และ guest_no เป็น primary key.

ให้สร้าง relational algebra สำหรับ queries ต่อไปนี้

2.1 แสดงรายชื่อของโรงแรมทั้งหมด

2.2 แสดงห้องเดี่ยวทั้งหมดที่มีราคาต่ำกว่า 2,400 บาทต่อคืน

2.3 แสดงชื่อและที่อยู่ทั้งหมดของแขก

2.4 แสดงราคาและประเภทของห้องพักทุกห้องของโรงแรม Crown Princess

2.5 แสดงรายชื่อของแขกที่กำลังเข้าพักอยู่ในโรงแรม Crown Princess

2.6 แสดงรายละเอียดทั้งหมดของห้องพักทุกห้องของโรงแรม Crown Princess รวมทั้งชื่อของแขกที่พักอยู่ในห้อง ในกรณีที่ห้องมีผู้พักอาศัยด้วย

2.7 แสดงรายละเอียดของแขก (guest_no, guest_name และ guest_address) ที่กำลังเข้าพักอยู่ในโรงแรม Crown Princess

3. ให้สร้าง tuple relational calculus สำหรับ queries ในแบบฝึกหัดข้อ 2.1-2.7
4. ให้สร้าง domain relation calculus สำหรับ queries ในแบบฝึกหัดข้อ 2.1-2.7
5. ให้ออกแบบรายงาน 3 รายงานสำหรับช่วยในการบริหารจัดการธุรกิจ "เช่า-ฉัน-สิ" ในภาคผนวก จากนั้นให้สร้าง relational algebra สำหรับ queries ที่ใช้เพื่อแสดงข้อมูลในรายงานนั้น โดยใช้ relational model ที่ได้ออกแบบไว้ในแบบฝึกหัดท้ายบทที่ 4

บรรณานุกรมและเอกสารอ้างอิงประจำบท

ชนวัฒน์ ศรีสอ้าน. (2542). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

ศุภกฤษฎิ์ นิวัฒนากูล. (2547). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

Connolly, T. and Begg, C. (2004). **Database Systems: A Practical Approach to Design, Implementation and Management, 4th ed.** Boston : Addison-Wesley.

Elmarsi, R. and Navathe, S. B. (2006). **Fundamentals of Database Systems, 5th ed.** Boston : Addison-Wesley.

Ramakrishnan, R. and Gehrke, J. (2003). **Database Management Systems, 3rd ed**. Singapore : McGraw-Hill.